

Examen Parcial – Programación 2

Tema: Programación Orientada a Objetos con una única clase

Consigna general

Desarrollar en Java una solución orientada a objetos que permita modelar una entidad del problema usando una única clase, aplicando los conceptos vistos hasta el momento.

Además, se deberá realizar una clase de prueba con `main` para demostrar el funcionamiento del programa.

Caso propuesto: Sistema de Gestión de Cursos

Se desea modelar una clase llamada `Curso`, que represente un curso dentro de una plataforma educativa.

Clase `Curso`

Atributos

La clase debe tener los siguientes atributos:

- `final int id` → identificador único del curso
- `String nombre`
- `int duracionHoras`
- `double precio`
- `static int totalCursos`

Requerimientos

1. Uso de constructores sobrecargados

La clase debe tener:

- Un constructor que reciba todos los datos necesarios para crear un curso.
- Otro constructor que reciba únicamente:
 - `nombre`
 - `duracionHoras`

En este segundo caso, el precio deberá tomar un valor por defecto de 100.

2. Uso de `this`

Se debe utilizar `this` para diferenciar atributos de parámetros dentro de constructores y métodos.

3. Encapsulamiento

Todos los atributos deben ser `private`.

Se deben crear únicamente los métodos `getter` y `setter` que sean necesarios.

No debe existir `setter` para:

- `id`
- `totalCursos`

4. Programación defensiva

La clase debe impedir que un objeto se cree o se modifique con datos inválidos.

Validaciones mínimas:

- `nombre` no puede ser `null` ni vacío.
- `duracionHoras` debe ser mayor que 0.
- `precio` no puede ser negativo.

5. Métodos sobrecargados

Implementar el método `actualizarPrecio` en dos versiones:

- una que reciba un porcentaje de aumento (`double`)
- otra que reciba un monto fijo de aumento (`int`)

Ambas versiones deben validar que el valor recibido no sea negativo.

6. Método estático

Implementar un método estático llamado:

```
mostrarTotalCursos()
```

que permita retornar la cantidad total de cursos creados.

7. Método `toString()`

Implementar el método `toString()` para mostrar de forma clara:

- `id`
- `nombre`
- `duración en horas`
- `precio`

Clase de prueba

Crear una clase `Main` que permita:

1. Instanciar al menos 3 objetos `Curso`, usando ambos constructores.
2. Aplicar las dos versiones de `actualizarPrecio`.
3. Mostrar por consola la información de cada objeto usando `toString()`.
4. Mostrar la cantidad total de cursos creados.
5. Incluir pruebas con datos inválidos para demostrar la robustez del programa.

Criterios de evaluación

Criterio	Puntaje
Definición correcta de atributos y clase	10
Uso correcto de constructores sobrecargados	15
Uso correcto de <code>this</code>	10
Encapsulamiento	15
Validaciones y programación defensiva	20
Sobrecarga de métodos	15
Uso de atributos y métodos estáticos	10
<code>toString()</code> y clase de prueba	5

6. **Total: 100 puntos**

Observaciones para el estudiante

- No se deben usar relaciones entre clases.
- Todo el desarrollo debe resolverse con una única clase de dominio y una clase de prueba.
- Se evaluará no solo que el programa funcione, sino también que esté bien diseñado según los conceptos de POO trabajados en clase.
- El video forma parte de la entrega y será tenido en cuenta para verificar la comprensión de la resolución presentada.

Ejercicio 2

A partir de un número entero N , impar y mayor a 3, desarrollar un programa en Java que genere una matriz de tamaño $N \times N$ con las siguientes características:

Consignas

- Validar que el valor de N sea:
 - Entero
 - Mayor a 3
 - Impar
- Crear una matriz cuadrada donde:
 - Todas las posiciones contengan valores enteros aleatorios.
 - La fila central y la columna central compartan los mismos valores aleatorios en sus posiciones correspondientes.
 - El valor ubicado en el centro de la matriz debe pertenecer tanto a la fila central como a la columna central.
- Obtener los valores de:
 - La fila central
 - La columna central

almacenándolos en **arreglos independientes**.

- Calcular el promedio de:
 - La fila central
 - La columna central
- Corroborar que ambos promedios sean iguales.
- El programa debe estar dividido en métodos obligatorios.

Generación de números aleatorios en Java

```
import java.util.Random;  
  
Random random = new Random();  
int numero = random.nextInt(100) + 1;
```

Ejemplo

Entrada:

$N = 5$

Matriz generada:

```
[12, 45, 80, 31, 20]
[7, 19, 15, 44, 60]
[80, 15, 30, 22, 41]
[55, 36, 22, 18, 73]
[9, 64, 41, 27, 11]
```

Fila central:

```
[80, 15, 30, 22, 41]
```

Columna central:

```
[80, 15, 30, 22, 41]
```



Salida esperada

```
Promedio de la fila central: 37.6
Promedio de la columna central: 37.6
```

Los promedios son iguales.

Observaciones para el estudiante

- No se deben usar relaciones entre clases.
- Todo el desarrollo debe resolverse con una única clase de dominio y una clase de prueba.
- Se debe utilizar una **matriz estática**, es decir, un arreglo bidimensional `int[][]`.
- Se evaluará no solo que el programa funcione, sino también que esté organizado en métodos y sea claro en su lógica.
- El video forma parte de la entrega y será tenido en cuenta para verificar la comprensión de la resolución presentada.

Entrega General:

La entrega del parcial deberá incluir:

1. Código fuente resuelto

Deberán entregar el código completo de los ejercicios resueltos:

Armados dos proyectos y resolver por separado. Entregar un zip con los dos ejercicios.